# INGRION

*A Permissioned Blockchain Protocol for Regulated Capital Markets*

Version 1.0 | February 2025

INGRION Blockchain Platform — For Developer & Institutional Use

© 2025 INGRION Blockchain Platform — Confidential

# 1. Abstract

INGRION is a permissioned, role-based blockchain protocol purpose-built for regulated capital markets. It provides a tamper-evident, cryptographically verifiable infrastructure for the complete lifecycle of equity issuance — from initial public offering (IPO) filing through secondary market trading, regulatory oversight, and corporate actions — all enforced on-chain without reliance on centralised intermediaries.

Built in Go and leveraging Ed25519 digital signatures, a BadgerDB world-state database, and a custom TCP peer-to-peer network, INGRION achieves 3-second deterministic block slots with a stake-weighted, round-robin proposer rotation. Its four-role permission model — User/Investor, Validator, Regulator, and Company — gates every transaction type at the execution layer, ensuring that only authorised participants may perform sensitive operations such as DRHP filing, RHP approval, account freezing, and share allocation.

This whitepaper describes the protocol's architecture, consensus mechanism, transaction palette, IPO lifecycle engine, regulatory enforcement layer, validator governance, cryptographic security model, networking stack, and desktop client integration.

# 2. Introduction & Problem Statement

## The Inefficiencies of Traditional Capital Markets

Traditional equity markets — particularly the IPO process — are plagued by structural inefficiencies rooted in opacity, manual intermediation, and fragmented record-keeping. Key problems include:

- Settlement delays of T+1 or T+2 due to centralised clearing infrastructure.
- Opaque share allocation processes that disadvantage retail investors relative to institutional participants.
- Prospectus document integrity risks, where DRHP and RHP documents can be altered after filing without a tamper-evident audit trail.
- Regulatory enforcement latency, where freezing a suspicious account or halting trading requires out-of-band intervention that is not immediately visible on the public ledger.
- Dividend and corporate action settlement delays caused by disconnected shareholder registers.

© 2025 INGRION Blockchain Platform — Confidential

- Validator and node operator accountability gaps in proof-of-stake networks that lack transparent slashing governance.

### The INGRION Approach

INGRION addresses each of these problems by encoding the full capital-markets workflow as on-chain transactions executed within a permissioned blockchain. Every material event — from DRHP submission to final allocation, from regulatory mandate issuance to account unfreeze — is a signed, hashed, irreversible ledger entry. The result is a single source of truth that all participants — investors, companies, regulators, and validators — can independently verify.

### Design Principles

- **Permissioned by design:** No anonymous participation. Every address carries an on-chain role enforced at transaction execution.
- **Immutability:** All transactions are cryptographically signed and chained via SHA-256 block hashes. No data can be altered retroactively.
- **Regulatory supremacy:** The Regulator role has on-chain authority to freeze accounts, reject filings, and issue mandates — all recorded transparently.
- **Deterministic finality:** Blocks finalise at height − 3. No probabilistic confirmation windows.
- **Operator sovereignty:** A single Go binary runs the complete node. No external dependencies beyond BadgerDB.

## 3. Architecture Overview

INGRION is implemented as a single Go binary that encapsulates the complete node stack: P2P networking, consensus engine, transaction execution, REST API, and a CLI. The following diagram describes the major subsystems and their interactions.

### Technology Stack

| Layer | Technology | Rationale |
|---|---|---|
| Runtime | Go 1.21+ | Single-binary deployment, excellent concurrency primitives, strong standard library. |

© 2025 INGRION Blockchain Platform — Confidential

| Layer | Technology | Rationale |
|---|---|---|
| World State DB | BadgerDB v3 | Embedded LSM-tree key-value store. Fast writes, ACID transactions, no external dependency. |
| Cryptography | Ed25519 (crypto/ed25519) | 32-byte compact keys, fast verification, high security margin. SHA-256 for all hashes. |
| P2P Network | Custom TCP (Go net) | Framed JSON messages over persistent TCP connections. Up to 50 peers. |
| REST API | gorilla/mux | Role-gated HTTP endpoints for clients. API-key authentication header. |
| Consensus | Slot-based round-robin | Deterministic proposer rotation. 3-second slot duration. Configurable in genesis.json. |
| Mempool | In-memory map + file persistence | Up to 50,000 transactions. Persisted to mempool.json on shutdown. |

## Node Roles

Every node running the INGRION binary operates in one of four roles determined by the private key supplied at startup and the role recorded on-chain at genesis or via an on-chain validator join transaction:

| Role | Capabilities |
|---|---|
| User / Investor | Send INR, bid in IPOs, buy/sell/transfer stocks on the secondary market, view portfolio and prospectus documents. |
| Validator | All User capabilities plus: propose and sign blocks, manage stake, vote on slash proposals. |
| Regulator | Freeze/flag/unfreeze accounts, reject DRHPs, upload and approve RHPs, issue mandates, access full analytics and audit trails. |
| Company | File DRHPs, initiate stocks, open and cancel IPOs, issue dividends and corporate actions, submit on-chain contracts. |

## State Model

© 2025 INGRION Blockchain Platform — Confidential

The world state is a key-value namespace in BadgerDB. Key prefixes segregate different entity types:

- **`account_<address>`** — Account object: balance, blocked balance, stake, nonce, stock holdings, role, participation metrics.
- **`block_<height>`** — Full block JSON including transactions and AllocationOps.
- **`vreg_<address>`** — ValidatorInfo: stake, join height, score, active status.
- **`rhp_<stock>`** — RHPMetadata: price band, quotas, bidding window, status.
- **`bid_<stock>_<address>`** — Bid object per investor per stock.
- **`alloc_<stock>`** — AllocationPlan including cutoff price, category quotas, and per-bidder decisions.
- **`slash_<id>`** — SlashProposal with voter list and status.
- **`mandate_<id>`** — Regulatory Mandate with type, target, and expiry.

# 4. Consensus Mechanism

## Slot-Based Deterministic Proposer Rotation

INGRION uses a slot-based consensus model inspired by proof-of-stake leader election, but adapted for a permissioned environment where the validator set is known and trusted. The key properties are:

- **3-second slot duration:** Each block slot is exactly 3 seconds wide, anchored to the genesis timestamp. Slot N is scheduled at genesis_time + N × 3s.
- **Deterministic proposer selection:** The proposer for slot N is validators[N mod len(validators)], where validators is sorted deterministically by address. This is computed identically by all nodes.
- **Empty slot timeout:** If the scheduled proposer does not broadcast a valid block within 3 seconds, other nodes increment the proposer index and try the next validator. This prevents single-validator failure from halting the chain.
- **Proposer cooldown:** A configurable cooldown (genesis.json: proposerCooldown) prevents the same validator from proposing consecutive slots in rapid succession, distributing load evenly.

## Block Production

When a validator's slot arrives, the following steps execute:

© 2025 INGRION Blockchain Platform — Confidential

1. The validator waits for its slot time (waitForNextSlot) by sleeping until genesis_time + height × slot_duration.
2. It assembles up to maxTxPerBlock transactions from the mempool, prioritising by nonce order per sender.
3. It appends any pending AllocationOps (IPO settlement operations) from active AllocationPlans.
4. It computes the StateRoot (SHA-256 of sorted account state) and ValidatorRoot (SHA-256 of sorted validator registry).
5. It signs the canonical block header bytes with its Ed25519 private key.
6. It computes the block hash as SHA-256 of the serialised block (with ProposerSig and BlockHash fields zeroed).
7. It broadcasts the block to all connected peers via the BLOCK P2P message.
8. It applies the block to its own state and stores it in BadgerDB.

## Block Validation

Receiving nodes validate each incoming block by verifying:

- The block height is exactly one greater than the current chain height.
- The prevHash matches the stored head block hash.
- The proposer is the expected validator for this height (same deterministic formula).
- The Ed25519 signature over the canonical header bytes is valid for the proposer's public key.
- The block hash matches the recomputed SHA-256.
- Each transaction's Ed25519 signature is valid for the sender's public key.
- Each transaction's nonce is exactly the sender's current on-chain nonce plus one.
- Each transaction's role-based permission is satisfied.

## Finality

INGRION uses a simple finality rule: a block at height H is considered finalised when the chain reaches height H + 3. The getFinalizedHeight() function returns current_height − 3. This provides a 9-second finality window (3 slots × 3 seconds) before a transaction is treated as irreversible by the application layer.

## State Snapshots

© 2025 INGRION Blockchain Platform — Confidential

To support fast node sync, the chain generates StateSnapshot objects every 1,000 blocks (SnapshotInterval). A snapshot contains the complete account set, validator registry, RHP metadata, and AllocationPlans at that height, allowing new nodes to bootstrap from a snapshot rather than replaying all blocks from genesis. Snapshots are stored in the data/snapshots/ directory as JSON files.

# 5. Transaction Palette

INGRION supports 26 distinct transaction types, each gated to one or more roles at execution time. Every transaction shares the same base structure:

*{ type, from, to, amountPaise, nonce, stock, bidPricePaise, bidShares, category, rhpHash, meta, timestamp, sig, shares, pricePaise, reason, mandateType, actionType, ratio, proposalId }*

The sig field is an Ed25519 signature over the compact JSON serialisation of the transaction with sig set to an empty string. The nonce must equal the sender's on-chain nonce + 1 to prevent replay attacks.

## Transaction Type Reference

| Transaction Type | Role | Description |
|---|---|---|
| tnx_sendINR | All | Transfer INR (in paise) from one address to another. Optional SHA-256 document hash may be attached in meta. |
| tnx_bid_stock | User | Place a bid in an active IPO bidding window. Category (QIB/NIB/Retail) is fixed at account setup. |
| tnx_buy_stock | User/Validator | Place a buy order on the secondary market at a specified max price per share. |
| tnx_sell_stock | User/Validator | Place a sell order on the secondary market at a specified asking price per share. |
| tnx_transfer_stock | User/Validator | Peer-to-peer transfer of shares to another address after listing. |
| tnx_upload_drhp | Company | File a Draft Red Herring Prospectus on-chain. Payload includes SHA-256 of the business plan document. |
| tnx_upload_rhp | Regulator | Upload the approved Red Herring Prospectus setting price band, quotas, and bidding window blocks. |

© 2025 INGRION Blockchain Platform — Confidential

| Transaction Type | Role | Description |
|---|---|---|
| tnx_update_rhp | Company | Amend RHP before the bidding window opens. |
| tnx_reject_drhp | Regulator | Formally reject a DRHP on-chain with a recorded reason. |
| tnx_initiate_stock | Company | Initiate the stock on-chain by referencing the approved RHP hash. |
| tnx_open_ipo | Company | Open the IPO bidding window, setting biddingStartSlot and biddingEndSlot on-chain. |
| tnx_cancel_ipo | Company | Cancel an IPO before or during the bidding window. Blocked balances are returned. |
| tnx_allocate_ipo | Chain (auto) | Chain-generated allocation transaction inserted after biddingEndSlot. Not submitted by users. |
| tnx_dividend | Company | Issue a dividend in paise per share to all current shareholders of a stock. |
| tnx_corporate_action | Company | Record a stock split, buyback, or bonus share event. Ratio field specifies split ratio (e.g. 2:1). |
| tnx_new_contract | Company | Submit an on-chain contract (name, batch, payload) pending regulator approval. |
| tnx_vote_contract | Regulator | Approve a pending on-chain contract by name. |
| tnx_freeze_account | Regulator | Freeze a suspicious address. Frozen accounts cannot send transactions. |
| tnx_unfreeze_account | Regulator | Lift a previously imposed account freeze. |
| tnx_flag_account | Regulator | Flag an address for investigation without freezing. Recorded on-chain with reason. |
| tnx_mandate | Regulator | Issue a regulatory mandate: trading halt, audit, freeze_ipo, or custom type. Optionally targeted at a stock or address. |
| VALIDATOR_JOIN | Validator | Register as an active validator with a stake deposit meeting the minStakePaise threshold. |
| VALIDATOR_EXIT | Validator | Deregister from the active validator set. Stake is returned to available balance. |
| tnx_update_stake | Validator | Top up or reduce stake without a full exit/rejoin cycle. |
| tnx_slash_proposal | Validator | Propose slashing a misbehaving validator by specifying the target, amount, and reason. |

© 2025 INGRION Blockchain Platform — Confidential

| Transaction Type | Role | Description |
|---|---|---|
| tnx_vote_slash | Validator | Vote yes on a pending slash proposal by its proposalId. |

# 6. IPO Lifecycle Protocol

INGRION's most distinctive feature is a fully on-chain IPO lifecycle with immutable audit trail. Every step from DRHP filing through final share allocation is executed as a signed transaction, creating an immutable record accessible to all participants.

## Lifecycle Stages

| Step | Transaction | Description |
|---|---|---|
| 1 | tnx_upload_drhp | Company files Draft Red Herring Prospectus. The SHA-256 of the business plan document is embedded in meta.payload, providing a tamper-evident anchor for the document. |
| 2 | tnx_upload_rhp | Regulator approves and uploads the Red Herring Prospectus, setting total shares, price band (upper/lower in paise), bidding window block count, face value, category quotas (QIB/NIB/Retail percentages), and lot size constraints. |
| 2a | tnx_update_rhp | Optional: Company may amend the RHP before the bidding window opens. |
| 2b | tnx_reject_drhp | Alternative: Regulator rejects the DRHP with a recorded reason. Company must refile. |
| 3 | tnx_initiate_stock | Company initiates the stock on-chain, referencing the rhpHash (auto-computed from the approved RHP payload). This anchors the stock to its approved prospectus. |
| 4 | tnx_open_ipo | Company opens the bidding window. biddingStartSlot and biddingEndSlot (block heights) are set on-chain. No bids are accepted outside this window. |
| 5 | tnx_bid_stock | Investors submit bids specifying bid price (within price band), number of shares, and category. Bid amounts are locked from the investor's available balance. |
| 6 | tnx_allocate_ipo | At biddingEndSlot + 1, the chain auto-generates an allocation transaction. No user submission required. |

© 2025 INGRION Blockchain Platform — Confidential

| Step | Transaction | Description |
|---|---|---|
| 7 | AllocationOps | Subsequent blocks embed AllocationOperation records that credit shares to winning bidders and refund losing bidders in paginated batches until allocation is complete. |
| 8+ | Post-listing | Company may issue dividends (tnx_dividend), corporate actions (tnx_corporate_action), and contracts (tnx_new_contract). Investors may trade freely on the secondary market. |

## Allocation Engine

The allocation engine is the most complex component of the INGRION protocol. It operates as follows when the bidding window closes:

### 1. Cutoff Price Determination

The engine collects all bids per category sorted by bid price descending. It walks the sorted bid list, accumulating shares until the category quota is filled. The last price at which the quota is exactly filled becomes the cutoff price for that category. Bids below the cutoff price are rejected; bids at or above are eligible for allocation.

### 2. Category Quota Enforcement

Total shares are partitioned by the QIB/NIB/Retail percentages specified in the RHP. For example, with 1,000,000 total shares and QIB: 50%, NIB: 15%, Retail: 35%, the quotas are 500,000, 150,000, and 350,000 shares respectively.

### 3. Oversubscription and Lottery

If a category is oversubscribed (total eligible bid shares exceed the quota), proportional allocation is applied for QIB and NIB. For the Retail category, a lottery mechanism (lotteryWin field in AllocationDecision) is used, consistent with real-world SEBI guidelines.

### 4. Lot Size Enforcement

Retail bids must conform to the lot size constraints (retailLotSize, minRetailBid, maxRetailBid) defined in the RHP. The execution engine validates these constraints at bid submission time.

### 5. Paginated Settlement

Because a large IPO may have thousands of bidders, AllocationOps are processed in paginated batches embedded across multiple blocks. The AllocCursor field in the block header tracks progress, and the allocPlan stores a Cursor integer that advances with each block until all allocations are settled. This prevents any single block from exceeding its gas limit.

© 2025 INGRION Blockchain Platform — Confidential

> *Key invariant: The total shares allocated + total shares refunded = total shares available. The chain enforces this mathematically at settlement time.*

# 7. Secondary Market

Once a stock reaches Completed status following IPO allocation, it enters the secondary market where holders may trade freely among themselves through three transaction types.

## Buy Orders (tnx_buy_stock)

An investor places a buy order specifying the stock symbol, number of shares, and a maximum price per share in paise. The execution engine checks that the investor's available balance is sufficient to cover the maximum total spend (shares × pricePaise). The order is matched against available sell orders on a price-time priority basis.

## Sell Orders (tnx_sell_stock)

A holder places a sell order specifying the stock, number of shares (capped at their current holding), and an asking price per share. The execution engine verifies the holding is sufficient. Matched buyers receive the shares; the seller receives pricePaise × shares credited to their balance.

## Peer-to-Peer Transfer (tnx_transfer_stock)

A direct stock transfer between two addresses at no exchange price. The sender specifies the recipient address, stock symbol, and number of shares. The execution engine verifies the sender holds sufficient shares and that the recipient address exists on-chain. Useful for off-market block trades and institutional transfers.

## Secondary Market Integrity

All secondary market transactions are gated by:

- Sufficient balance or stock holding verification at execution time.
- Nonce replay protection — each transaction must increment the sender's nonce by exactly one.
- Ed25519 signature verification — the transaction must be signed by the private key corresponding to the from address.

© 2025 INGRION Blockchain Platform — Confidential

- Mandate check — a trading_halt mandate issued by the Regulator targeting the stock blocks all secondary market transactions for that symbol.

# 8. Regulatory Enforcement Layer

INGRION places regulatory authority on equal footing with the cryptographic protocol itself. Regulatory actions are not out-of-band administrative decisions — they are signed, chained, and irreversible on-chain transactions that any participant can audit.

## Account Enforcement

### Freeze Account (tnx_freeze_account)

The Regulator may freeze any address. A frozen account cannot submit any transaction. The freeze is recorded with the reason text and the block height at which it was imposed. Multiple independent freeze events on the same address are each recorded separately.

### Unfreeze Account (tnx_unfreeze_account)

Only the Regulator may lift a freeze. The unfreeze is also recorded on-chain, creating a complete enforcement lifecycle log.

### Flag Account (tnx_flag_account)

A softer enforcement tool that marks an address for investigation without restricting its transactions. The flag reason and block height are recorded. Regulators may escalate a flagged account to frozen via a subsequent tnx_freeze_account.

## Mandates (tnx_mandate)

Mandates are powerful regulatory directives that can affect the entire chain or specific targets:

- **trading_halt:** Blocks all buy, sell, and transfer transactions for the targeted stock symbol.
- **audit:** Marks an address or stock for active regulatory audit. Informational; does not restrict transactions.
- **freeze_ipo:** Prevents the opening of the IPO bidding window for the targeted stock.
- **Custom types:** Extensible mandate type field allows future regulatory directive types without protocol upgrades.

© 2025 INGRION Blockchain Platform — Confidential

Mandates record the issuer address, target (stock symbol or address, or global if blank), reason, issuing block height, and an optional expiry block height (expiresAt = 0 means indefinite).

### Prospectus Oversight (DRHP/RHP Review)

The Regulator is the sole role authorised to upload an RHP or reject a DRHP. This creates a mandatory regulatory checkpoint in the IPO lifecycle. The RHP upload transaction embeds the SHA-256 hash of the analysis report document, providing cryptographic evidence of the exact document the Regulator reviewed.

### Contract Approval (tnx_vote_contract)

On-chain contracts submitted by companies require Regulator approval before they take effect. The tnx_vote_contract transaction records the approval by contract name, creating an auditable approval chain.

> *All regulatory transactions are permanently recorded on-chain with the Regulator's Ed25519 signature. No regulatory action can be taken, altered, or concealed without leaving a cryptographic footprint.*

## 9. Validator & Staking Governance

### Validator Set Management

Validators are the block-producing nodes of the INGRION network. The initial validator set is defined in genesis.json. New validators may join dynamically via VALIDATOR_JOIN and exit via VALIDATOR_EXIT.

| Parameter | Details |
|---|---|
| Minimum stake | minStakePaise (set in genesis.json). Joining with less than this amount is rejected. |
| Stake storage | Stake is locked in the validator's stakePaise field. It is excluded from available balance. |
| Stake update | tnx_update_stake allows incremental top-up or reduction without a full exit/rejoin. |

© 2025 INGRION Blockchain Platform — Confidential

| Parameter | Details |
|---|---|
| Exit | VALIDATOR_EXIT deactivates the validator and returns stake to available balance. The validator is excluded from future proposer rotation immediately. |
| Scoring | Each validator carries an overallScore and participation metric updated on each block. These are used to rank validators in analytics dashboards. |

## Proposer Rotation

Proposer selection is entirely deterministic: the validator at index (height mod len(validators)) in the address-sorted active validator list proposes block height. All nodes compute this independently, requiring no coordination message for leader election.

## Slash Governance

INGRION implements an on-chain slash governance mechanism that allows validators to propose and vote on slashing penalties for misbehaving peers:

### Slash Proposal (tnx_slash_proposal)

Any active validator may propose slashing another validator by specifying the target address, the slashPaise amount, and a reason. A new SlashProposal record is created on-chain with status: pending.

### Slash Voting (tnx_vote_slash)

Other validators vote yes on a pending proposal by submitting tnx_vote_slash with the proposalId. Votes are recorded in the proposal's votes array. When a quorum of active validators has voted yes, the proposal status transitions to approved and the slash amount is deducted from the target validator's stake.

### Proposal Outcomes

If insufficient votes are received before the proposal's implicit timeout (tracked by block height), the proposal transitions to rejected. All proposals, votes, and outcomes are permanently stored on-chain and visible to all participants.

# 10. Document Integrity — DRHP & RHP Hashing

© 2025 INGRION Blockchain Platform — Confidential

One of INGRION's most practically valuable features is its on-chain document integrity layer. Prospectus documents — which form the legal basis for investor decisions — are anchored to the chain via SHA-256 hashes at the time of filing.

### Hash Computation

The SHA-256 hash is computed over the raw bytes of the document file (for PDF files) or over the compact JSON serialisation of the document object (for JSON business plans and analysis reports). This ensures that the hash is deterministic and can be reproduced from the original document by any party.

### On-Chain Anchoring

The computed hash is embedded in the meta.payload field of the relevant transaction:

- tnx_upload_drhp embeds the SHA-256 of the company's business plan document.
- tnx_upload_rhp embeds the SHA-256 of the regulator's analysis report.
- tnx_sendINR optionally embeds the SHA-256 of any attached document in the transaction meta, allowing payment-linked document authentication.

### Verification Protocol

Any party in possession of the original document can verify its integrity by:

9. Computing the SHA-256 hash of the document locally.
10. Querying the chain (GET /api/rhp/{stock}/status) to retrieve the on-chain hash.
11. Comparing the two hashes. A match proves the document has not been altered since filing.

> *Because the hash is stored as part of a signed, chained block transaction, neither the company nor the regulator can retroactively alter a filed document without invalidating all subsequent block hashes — a cryptographically impossible operation.*

## 11. Networking & Synchronisation

### P2P Protocol

INGRION uses a custom TCP-based P2P protocol with length-prefixed, framed JSON messages. The maximum message size is 10 MB. Up to 50 peer connections are maintained simultaneously.

| Message Type | Purpose |
|---|---|
| HELLO | Initial handshake. Sends nodeId, address, height, headHash, genesisHash, stateRoot, validatorRoot, finalizedHeight, and known peer list. Nodes with mismatched genesis hash are rejected. |
| META | Exchange of NodeMeta: stake, participation score, known peers, last proposed block. |
| BLOCK | Broadcast a newly proposed block to all peers. |
| GETBLOCK | Request a single block by height. |
| GETRANGE | Request a range of blocks [start, end] for batch sync. Batch size: 50 blocks. |
| GETSTATE | Request the current ChainState snapshot from a peer. |
| STATESNAP | Response to GETSTATE: full StateSnapshot enabling fast-sync of a new node. |
| TX | Broadcast a transaction to all peers for mempool propagation. |
| PING/PONG | Keep-alive messages sent every 30 seconds. Peers not responding within 5 minutes are pruned. |

## Peer Discovery

Bootstrap peers are defined in config_peers.json and loaded at startup. On HELLO exchange, each peer shares its known peer list (KnownPeers), enabling organic peer discovery. The whitelist.json file restricts which addresses may connect, implementing the permissioned network boundary.

## Sync Strategy

A new node joining the network follows this sync sequence:

12. Connect to bootstrap peers and complete HELLO handshake.
13. If local DB is empty: send GETSTATE to the peer with the highest reported height. Receive STATESNAP with full account and validator state.
14. Send GETRANGE requests in batches of 50 blocks from block 1 to the peer's current height.
15. Apply each block sequentially, verifying header and proposer signature.

© 2025 INGRION Blockchain Platform — Confidential

16. Once caught up, switch to ongoing sync mode: poll GET /status every 5–10 seconds and fetch new blocks as they appear.

### Error Handling & Offline Resilience

If the node is unreachable, the sync worker backs off exponentially (5s → 10s → 30s → 60s). All operations that require live node access return cached data with a clear indication that the data may be stale. The client application displays a prominent 'Node Offline — Cached Data' banner in all offline states.

## 12. Security Model

### Cryptographic Foundations

Every participant identity and every transaction in INGRION is secured by Ed25519 elliptic-curve cryptography:

- **Key generation:** 32-byte Ed25519 seed generates a 64-byte keypair. The 32-byte public key is the on-chain address (hex-encoded).
- **Transaction signing:** Every transaction is signed with the sender's Ed25519 private key over the compact JSON serialisation (sig field set to empty string before signing).
- **Block signing:** The proposing validator signs the canonical block header bytes. The signature is stored in ProposerSig and verified by all receiving peers.
- **Hash chaining:** Each block header includes the SHA-256 hash of the previous block (prevHash), creating a tamper-evident chain. Altering any historical block invalidates all subsequent block hashes.

### Replay Protection

Every transaction carries a nonce field that must equal the sender's current on-chain nonce plus exactly one. Transactions with incorrect nonces are rejected at mempool entry. This prevents replay attacks where a valid signed transaction is rebroadcast by a third party.

### Permission Enforcement

Role-based access control is enforced at transaction execution time, not merely at the API layer. When a block is applied to the world state, the executor checks the from address's on-

© 2025 INGRION Blockchain Platform — Confidential

chain role against the required role for each transaction type. A User address submitting tnx_freeze_account will cause the entire block to be rejected.

### API Authentication

The REST API requires an X-API-Key header on all authenticated endpoints and an X-Caller-Address header on transaction submission. The API key is configured at node startup and stored in memory only. The X-Caller-Address header allows the node to enforce that the transaction's from field matches the calling client.

### Whitelist Network Boundary

The P2P layer enforces a whitelist of permitted peer addresses (whitelist.json). Connection attempts from non-whitelisted addresses are rejected at the TCP accept level, ensuring that only authorised nodes participate in the network.

### Private Key Management (Desktop Client)

The desktop client stores the user's Ed25519 private key in an encrypted keystore file ({AppData}/INGRION/keystore.json) using AES-256-GCM authenticated encryption. The encryption key is derived from the user's password via PBKDF2 with 600,000 iterations and SHA-256. The raw private key is never written to disk. It is decrypted in memory only at transaction signing time and wiped immediately afterward.

## 13. Token Economics

### INR-Paise Denomination

INGRION's native currency unit is the paise, the smallest denomination of the Indian Rupee. All monetary fields in the protocol are stored and transmitted as int64 paise values. The display layer converts paise to INR by dividing by 100. This eliminates floating-point precision issues from all financial calculations.

### Genesis Distribution

The initial account balances are defined in genesis.json under the initialAccounts array. Each entry specifies the address, balancePaise, and role. There is no mining or inflationary token emission — all supply is defined at genesis.

© 2025 INGRION Blockchain Platform — Confidential

### Gas Model

Each transaction consumes a fixed gas cost defined by the gasPerTx genesis parameter. Gas is deducted from the transaction sender's balance at execution time. Empty blocks (no transactions) consume no gas. This simple fixed-gas model avoids the complexity of EVM-style variable gas markets.

### Staking Economics

Validators are required to stake at least minStakePaise to participate in block production. Staked amounts are locked and excluded from available balance. The protocol records staking rewards via the GET /api/stake/rewards/{address} endpoint, though the specific reward distribution formula is configurable per deployment.

### Dividends

After listing, companies may issue dividends via tnx_dividend, specifying an amountPaise per share. The execution engine distributes the dividend to all current shareholders proportional to their holdings, crediting paise to each holder's balance and debiting from the company's balance.

### Corporate Actions

Stock splits (ratio field, e.g. 2:1), buybacks (amountPaise per share), and bonus share issuances are recorded on-chain via tnx_corporate_action. These events provide a permanent, auditable record of all changes to the share structure of a listed company.

## 14. Desktop Client Integration

INGRION ships a full-featured desktop application built on Tauri (Rust backend) and React 18 (TypeScript). The client interfaces with the node exclusively via the REST API documented in Appendix A, and stores analytics, transaction history, and user preferences in a local SQLite database.

### Role-Locked UI

The client reads the user's role from the encrypted keystore and renders a completely different navigation structure and page set for each role. Navigation items, transaction forms, and dashboard zones are all gated at the UI layer, mirroring the on-chain permission model.

© 2025 INGRION Blockchain Platform — Confidential

## Background Sync Service

A continuous background worker syncs new blocks from the node and indexes all transactions into the local SQLite database. This enables the analytics dashboard, transaction history, and notification system to operate using locally computed data without repeated RPC calls. The sync worker polls every 5–10 seconds and fetches new blocks via GETRANGE when the chain has advanced.

## Local Analytics

All analytics charts on the Regulator dashboard are computed from the local SQLite block cache, not from live RPC calls. An aggregation worker runs hourly to compute daily volume, active addresses, validator participation, and holder concentration metrics. Holder snapshots (including Gini coefficient computation) run weekly.

## Document Hash Tool

A built-in document hash tool (available on all pages as a floating action button) computes SHA-256 hashes of PDF and JSON files entirely client-side using the Web Crypto API. The Verify tab allows users to compare a locally computed hash against an on-chain DRHP or RHP hash, confirming document integrity without any server interaction.

## REST API Reference Summary

| Endpoint | Method | Usage |
|---|---|---|
| GET /status | Public | Node height, chainId, mempool size, sync state. |
| GET /api/balance/{address} | API Key | Account state: balance, nonce, role, stock holdings. |
| GET /api/rhp/{stock}/status | API Key | RHPMetadata for a stock: price band, quotas, bidding window, status. |
| GET /api/ipo/{stock}/bids | API Key + Role | Full bid orderbook for a stock in bidding status. |
| GET /api/frozen-accounts | Regulator | List of all currently frozen addresses. |
| GET /api/audit/{address} | Regulator | Full audit history: flag events, freeze history, tx patterns. |
| GET /api/mandate/active | Regulator | All currently active regulatory mandates. |
| GET /api/contracts | Regulator | All on-chain contracts (pending and approved). |

© 2025 INGRION Blockchain Platform — Confidential

| Endpoint | Method | Usage |
|---|---|---|
| GET /api/stocks/holders/{stock} | Company/Reg | Shareholder register for a listed stock. |
| GET /validators | API Key | Active validator set with stake and score. |
| POST /api/submitTx | API Key | Submit a signed transaction for inclusion in the next block. |

# 15. Roadmap & Conclusion

## Current State (v1.0)

INGRION v1.0 delivers a fully functional permissioned blockchain node with:

- Complete IPO lifecycle from DRHP filing through paginated allocation settlement.
- 26 transaction types covering all four role domains.
- 3-second deterministic slot consensus with empty slot tolerance.
- Full regulatory enforcement layer (freeze, flag, mandate, reject).
- Validator governance with stake management and slash proposals.
- TCP P2P networking with HELLO handshake, batch sync, fast-sync snapshot, and keep-alive.
- REST API with role-gated endpoints and API key authentication.
- Desktop client with role-locked UI, local analytics, and encrypted keystore.

## Future Development

- **Multi-sig regulatory mandates:** Require M-of-N regulator signatures for high-impact actions such as global trading halts.
- **Cross-chain bridge:** Bridge INR-backed stablecoins between INGRION and public EVM chains.
- **Zero-knowledge compliance proofs:** Allow investors to prove KYC compliance without revealing identity on-chain.
- **Automated market maker:** On-chain AMM liquidity pools for listed stocks to supplement the order-book secondary market.
- **Mobile light client:** SPV-style mobile application for investor portfolio monitoring and transaction signing.

© 2025 INGRION Blockchain Platform — Confidential

- **Auto-update infrastructure:** Tauri updater integration for seamless desktop client version management.

## Conclusion

INGRION represents a principled application of blockchain technology to a domain where its properties — immutability, cryptographic auditability, deterministic execution, and permissioned access — deliver genuine value. By encoding the entire capital markets workflow as on-chain transactions, INGRION eliminates the opacity, delays, and intermediation costs that characterise traditional IPO and secondary market infrastructure.

The protocol's four-role permission model ensures that every participant operates within their authorised domain, while the Regulator role provides the enforcement authority that permissioned capital markets require. The result is a system where investors, companies, and regulators share a single, verifiable source of truth — one that neither party can unilaterally alter, conceal, or dispute.

INGRION is designed to be run, audited, and built upon. Its single-binary deployment, well-documented REST API, and open transaction schema make it accessible to developers, regulators, and institutional participants alike.

*— End of INGRION Whitepaper v1.0 —*

© 2025 INGRION Blockchain Platform. For Developer & Institutional Use.